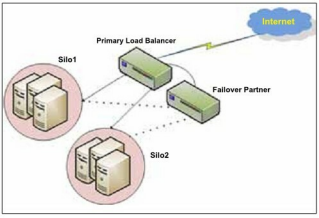

Figure 1: Typical load balancer set-up

In the technical sense, a load balancer balances underlying TCP connections, rather than actual Web requests. It is a misconception that a load balancer checks resource utilisation (such as CPU, memory, etc) on a controlled server. In reality, it simply checks the network response time of a server, which is a result of the server's overall resource utilisation. Since it acts as a catalyst in improving the scalability of a server farm, it maintains data for each node under its control, like the number of requests processed in history, the response time by each host for requests, the fault trend of each host, etc. In earlier days, load balancing solutions were implemented around simple round-robin techniques, which did help distribute the load, but did not provide fault tolerance features, since they lacked the necessary intelligence.

In today's advanced data centres, load balancers are used to effectively distribute traffic for Web servers, databases, queue managers, DNS servers, email and SMTP traffic, and almost all applications which use IP traffic. Balancing DNS servers helps distribute DNS queries to servers that are dispersed geographically, which is useful for disaster-recovery implementations.

Using load balancers to achieve fault tolerance

In a server farm, servers often experience downtime due to unforeseen resource failure or scheduled maintenance. These resource failures can be at the hardware level or simply at the software application level. In a business-critical infrastructure, such situations should be transparent, never affecting the user. As discussed earlier, since the balancing device maintains separate TCP connectivity with the controlled node, it can be further used to achieve fault tolerance.

A configurable 'heart beat', called a monitor, is maintained by the balancer with each node. This can be a simple ICMP ping, or an FTP/HTTP connection to retrieve data. Upon an appropriate response from the node, the load balancer becomes aware that the node is live, and marks it as an active participant eligible for the balancing process. If the server or its application resource fails, the balancer waits for a certain period of time for a 'heart beat' from the node; upon non-compliance, it marks that node as a non-participant, and removes it from the silo. Once marked thus, the load balancer doesn't send traffic to that node. However, it still keeps polling to see if the node is back online, and if found to be so, marks the node as an active participant again and starts sending traffic to it. If a fault situation occurs while the request is being transferred to a node, modern load balancers are capable of detecting that too, and taking (configurable) corrective action.

This feature can further be explored by the operations team for maintenance purposes. A service instance can be configured on a node—for example, a separate Web instance running under a separate IP address, with a dummy page on it. A monitor can be configured to access that page periodically. If the server is to be taken offline for maintenance purposes, the operations person can stop the dummy site, which results in the server being marked as a non-participant. It can then be shut down or have other administration work done on it. Once maintenance is completed, the dummy site service can be started again, bringing the server back into the silo. This feature can be further extended by configuring many such monitors at the application level that can be reported upon in a dashboard via a network monitoring product, for an operations admin view.

Layer 7 load balancing

Earlier versions of load balancers used to work at the OSI model Layer 2 (link aggregation), or Layer 4 (IP-based). Since the requests flow through the balancing devices, it made sense to read into the requests at Layer 7, to bring additional flexibility in balancing techniques. Adding such flexibility offers higher scalability, better manageability and high availability. Layer 7 load balancing primarily operates on the following three techniques:

- URL parsing
- HTTP header interception
- Cookie interception

Typically, a Layer 7 rule structure looks somewhat like the one shown below. However, the exact syntax varies for each vendor and device model. As seen in the example, a request is first parsed based on the virtual directory being accessed, then by a particular cookie field's content, and is finally sent to a default pool, if the first two conditions are not matched.

```
{
if (http_qstring, "/") = "mydir"
   sendto server_pool1
else{
if cookie("mycookie") contains "logon_cookie"
   sendto server_pool2
else {
   sendto server_pool3
}
```